

Zmail and CallChat 1.0: Layered Protected Communications Across Email, Messaging, and Calls

Shafaet Brady Hussain Independent researcher and developer, United Kingdom 18 August 2026

Abstract

This paper consolidates the protected-communications architecture I developed across Zmail and CallChat. The central contribution is not a new cipher but a layered application design using established primitives and protocols. Zmail protected mail creates a signed, dual-AES-GCM envelope for ready local browser devices, wraps content keys per device using ephemeral P-256 ECDH and HKDF-SHA-256, and leaves unsupported recipients on explicit standard-email paths. CallChat integrates Matrix room encryption, portable ZME1 object protection, local profile exchange, and a media-key bridge that mixes rotating MatrixRTC keys with a room-scoped local factor before LiveKit encoded-frame encryption.

I evaluated frozen local source snapshots on 18 August 2026. Twelve targeted Zmail/ZMath tests passed, including six protected-mail tests and six browser-vault tests. Two CallChat profile/interception programs passed, and a media verification program reported six passed contracts covering launch gating, packaged configuration, permission separation, key mixing, encoded-frame wiring, and domain separation. These author-run tests support narrow implementation claims; they do not establish independent security validation, universal end-to-end email encryption, secure endpoints, verified production parity, post-quantum security, or protection of ordinary mail and Matrix metadata.

Keywords: protected email, Matrix, end-to-end encryption, WebRTC, AES-GCM, ECDH, ECDSA, HKDF, fail-closed routing

1. Research question

The research question is: **How can one communications ecosystem apply additional object, device, and media protection without obscuring the different trust assumptions of email, Matrix messaging, and real-time calls?**

The answer is a layer model with explicit downgrade rules. A product can add cryptographic protection only where participants, keys, and client capabilities are known. Unsupported paths must remain visible rather than being described as protected. The scientific value is therefore in boundaries, state transitions, negative tests, and evidence - not in multiplying cipher names.

This paper contributes:

1. a common layer model for Zmail and CallChat;
2. a precise Zmail protected-mail envelope description;

3. a CallChat model separating Matrix transport, portable ZME1 objects, profile exchange, and media-key mixing;
4. a dated evaluation with source hashes and negative results;
5. a claim-risk ledger and falsifiable conformance programme; and
6. a public reproducibility companion that omits credentials, user content, server topology, and private policy material.

2. Evidence boundary

Primary evidence came from the public-facing Zmail platform source snapshot, the CallChat public-web snapshot, their targeted tests, and local documentation. I processed no live messages, calls, mailboxes, keys, accounts, or customer data. I did not publish private entitlement code, production configuration, recovery material, or deployment credentials.

The principal Zmail cryptographic module had SHA-256 01c6442ac4080a64a8641ea657cac872f43514c1a9822b11b5ba1eb7287de7d9; its protected-mail test had SHA-256 f0039be7b67da0a52f3a1e48aa161c57f1fdf2a6dd009e05f286e39da1f9bb19. The CallChat media verifier had SHA-256 ed300069eb51fed6c78e7d302f7e94d332e2ea9ad5287e83f73f7f2fcaff2e00. The evaluated ZMath Auto module and test had hashes 0f614659ef3e07b90d772057ca4ffadba5808b8c67c5d1c91e9d318bc230802a and d1938e8895c84fc38351fcdd2bf77f17ccf8c2900b16406d937abd9ccf3245ed.

These are file identities, not proof of deployment identity. The CallChat media verifier was run without its optional --live argument, so this paper does not claim that hosted assets matched the snapshot on 18 August.

3. Layer model

Let a communication attempt be

$M = (A, I, K, T, O, P, R)$

where A is application state, I identity and device state, K available key material, T transport protection, O optional object protection, P policy, and R the release decision. The model forbids treating one layer as evidence for another.

Layer	Zmail	CallChat	Boundary
Account and transport	Webmail, SMTP/JMAP/HTTPS context	Matrix homeserver, HTTPS, MatrixRTC	Servers still observe routing metadata.
Native content protection	Protected-mail envelope for ready local devices	Matrix Olm/Megolm room encryption	Only applies when the relevant path is enabled and verified.
Object protection	Protected message body; drafts/attachments excluded in evaluated v1	ZME1 files/messages as an additional portable envelope	Visible metadata and client capability remain separate.
Device/profile	Browser P-256 key pairs and fingerprints	Local ZMath profile and room-scoped factor exchange	Directory or exchange integrity is a trust assumption.
Real-time media	Not applicable	MatrixRTC key plus ZMath factor mixed for encoded-frame E2EE	Does not conceal call signalling or prove participant identity.
Application policy	Auto, Require ZMath, Standard	Require bridge/module before protected launch	Fail closed only in explicitly required modes.

4. Zmail protected mail

4.1 Device state

The evaluated browser creates one P-256 ECDH encryption key pair and one P-256 ECDSA signing key pair per browser profile. Private keys are non-exportable Web Crypto keys stored locally in IndexedDB. Public keys are registered through an authenticated Roundcube action. A device fingerprint is displayed, but the current design does not prove that the server directory cannot substitute keys. Manual verification, transparency, or key pinning remains future work.

4.2 Envelope construction

For a set of ready recipient devices, the sender creates two fresh AES-256-GCM content keys. The plaintext message body is encrypted inside an inner layer; the resulting inner object is encrypted again inside an outer layer. Each content key is wrapped separately to each device using an ephemeral P-256 ECDH shared secret and HKDF-SHA-256. The envelope is signed with the sender device's P-256 ECDSA key.

The two AES layers are independent application layers, not “AES-512,” and their benefit must not be assumed without analysis. The stronger architectural contribution is per-device wrapping, authenticated structure, explicit sender-key verification, and fail-closed recipient selection.

Version 2 uses opaque recipient slots. The shared protected envelope omits clear recipient and BCC addresses and does not expose device identifiers in the tested serialisation. This does not hide SMTP envelope recipients, message delivery timing, server logs, IP addresses, mailbox metadata, or headers outside the protected body.

4.3 Routing and downgrade

The composer exposes Auto, Require ZMath, and Standard policies. Require ZMath aborts if any target is not eligible. Auto protects only when every recipient is a ready local @zmail.my webmail device; otherwise it remains ordinary email and must say so. Standard is a deliberate per-message choice.

The evaluated version does not protect subject lines, address headers, drafts, or attachments. Drafts remain standard mailbox data. External recipients and not-yet-ready local recipients do not receive the protected envelope. Therefore “Zmail is universally end-to-end encrypted” is false.

4.4 Signature semantics

The ECDSA signature authenticates the envelope relative to the included sender public key and recomputed fingerprint. It does not independently authenticate a human identity unless that key-to-person binding has been verified. A malicious directory server capable of substituting keys is outside the current trust model.

5. CallChat protected messaging

CallChat builds on the Matrix ecosystem. Matrix Olm and Megolm provide the room-encryption protocol implemented by the client stack; I do not claim authorship of those protocols. Device verification, backup configuration, room encryption state, and client behaviour determine the actual protection level.

ZME1/ZShield adds portable authenticated object protection for files or short messages. This layer can keep plaintext outside Matrix attachment storage, but it carries its own passphrase/factor and metadata assumptions. It is additive: a ZME1 object can travel inside a Matrix-encrypted room, while ordinary Matrix clients may only see/download the protected object.

The ZMath Auto module intercepts supported UI flows and applies local factor/profile handling. Its tests cover separate-factor recovery and local-processing contracts. A room-share program covers profile ECDH exchange. Passing those tests does not prove that every Matrix event or third-party client follows the same path.

6. CallChat protected calls

The evaluated media architecture derives a room factor from a local media root using HKDF-SHA-256 with room identity in the salt. It then mixes the rotating MatrixRTC media key with that factor using a distinct domain string plus participant identifier and ratchet index. The result is a 256-bit key supplied to the LiveKit encoded-frame E2EE key provider.

The verification program checks that:

- voice, video, and screen-share launch paths require the bridge markers;
- the MatrixRTC timing configuration is packaged as JSON;
- microphone and camera permissions are requested independently;
- MatrixRTC key changes and room factors affect the mixed key;
- the custom key provider is wired to encoded-frame E2EE; and
- room, profile, Matrix key, participant, and ratchet index separate derived results.

HKDF key mixing does not create entropy or quantum security. The design does not prove the origin of the local factor, attendee identity, signalling confidentiality, secure TURN operation, or production asset parity. It must not be marketed as “quantum encrypted calling.”

7. Evaluation

7.1 Method and results

I ran targeted local tests on 18 August 2026 using Node.js. Zmail's protected-mail suite passed six tests: dual-layer round trip and sender signature, non-recipient refusal, tamper rejection, opaque recipient/BCC slots, forged fingerprint-label rejection, and fail-closed Roundcube integration. The ZMath browser-vault suite passed six further tests: note/attachment round trip, secret absence from exported JSON, wrong-factor/tamper rejection, weak-parameter rejection, absence of enumerated network/persistent-storage primitives, and visual-pattern controller behaviour.

Two CallChat programs printed success for the Element interception contract and room-profile ECDH exchange. The media verifier printed six PASS lines for the contracts listed in Section 6. No live parity flag was supplied.

Test group	Observed result	Proper interpretation
Zmail protected mail	6/6 tests passed	Same-codebase envelope and integration regression.
ZMath browser vault	6/6 tests passed	Local protected-artifact regression, not mail transport evidence.
CallChat profile/interception	2 programs passed	Selected UI/profile contracts only.
CallChat media verifier	6 reported contracts passed	Static/package and deterministic key-mixing checks; not a live call.

7.2 Negative evidence

The most important evidence is bounded failure: a non-recipient device is refused; changed outer ciphertext is rejected; a forged fingerprint label is rejected even when re-signed; Require ZMath has a server-side abort path; wrong vault inputs fail; and differing room, profile, Matrix key, participant, or ratchet values produce different derived media keys in the test.

7.3 What was not tested

No independent implementation, browser matrix, mobile matrix, fuzzing campaign, formal proof, live SMTP capture, live homeserver inspection, production call, packet trace, TURN audit, endpoint-compromise test, accessibility study, or third-party cryptographic review was performed. The static “no network primitive” scan is not a whole-application non-exfiltration proof.

8. Threat model

Principal adversaries include a mailbox or Matrix service observing metadata; an attacker obtaining protected envelopes for offline analysis; an unverified directory replacing public keys; a malicious or compromised recipient endpoint; modified ciphertext; an unsupported client; and a user misreading a UI badge as full-path assurance.

Residual risks include:

- compromised endpoints, browser extensions, screen capture, and keyloggers;
- weak local device access controls and browser-profile loss;
- metadata exposure through SMTP, Matrix, timing, membership, and attachment presence;
- directory key substitution before fingerprints are independently verified;
- plaintext drafts, subjects, attachments, or notifications outside protected scope;
- replay, rollback, backup, and multi-device lifecycle errors;
- copied plaintext after successful release;
- traffic analysis and service availability; and
- divergence between source snapshots and deployed assets.

9. Conformance and falsification

Zmail Protected Mail conformant means the client publishes an envelope profile; wraps keys only to declared ready devices; verifies signature and fingerprint; hides recipient slots inside the shared protected object; exposes downgrade status; and fails closed in required mode. **CallChat Protected Messaging conformant** additionally requires Matrix encryption state to be reported separately from ZME1 state. **CallChat Protected Media conformant** requires a declared rotating MatrixRTC key source, domain-separated factor mixing, encoded-frame E2EE wiring, and refusal when required protection material is unavailable.

Claims are falsified if: an ineligible device decrypts; a modified signed/authenticated field releases plaintext; Require ZMath sends standard mail; clear BCC addresses appear in the protected serialisation; identical derived media keys appear after changing a declared separation input; or a protected-call UI launches without required bridge material.

Future work should publish at least two independent envelope implementations, deterministic vectors, directory-transparency experiments, device-loss/revocation tests, cross-browser performance, attachment/subject policy, live packet-boundary evidence, and a reproducible production-parity check.

10. Claim ledger

Claim	Status
Zmail implements a tested protected-body envelope for ready local devices.	Supported for frozen snapshot.
Zmail protects all email, subjects, drafts, attachments, and external recipients.	Rejected.
Two AES layers equal AES-512 or prove twice the strength.	Rejected.
CallChat integrates Matrix E2EE and additional ZME1 object protection.	Supported as architecture; deployment state not reverified here.
The media snapshot contains deterministic domain-separated key-mixing logic.	Supported by local verifier.
A local factor makes calls quantum encrypted or post-quantum secure.	Rejected.
Tests constitute certification or independent audit.	Rejected.
Sender signatures prove human identity without key verification.	Rejected.

11. Limitations

This is an author-led implementation evaluation with an inherent conflict of interest. File hashes bind observations to snapshots but do not establish provenance or deployment identity. Same-codebase tests may repeat the implementation's assumptions. The CallChat media test inspected built artifacts and deterministic derivation but did not execute a live call or use its optional live parity mode. Zmail tests used synthetic accounts and messages.

Neither product can protect plaintext on a compromised endpoint. Encryption does not remove communication metadata. Classical P-256, AES-GCM, PBKDF2, HKDF, ECDSA, and Matrix protocols are established technologies, not inventions claimed here. No post-quantum KEM is evaluated in this paper.

12. Conclusion

Zmail and CallChat demonstrate a coherent boundary-first approach to protected communications: protect only where keys and capabilities are known, label downgrade paths, keep portable objects distinct from transport encryption, and block required modes when evidence is missing. The dated tests provide useful engineering evidence and negative cases, but not certification.

The next credible step is independent interoperability and live boundary testing: a second Zmail implementation, key-directory transparency, frozen cross-client vectors, and production-parity evidence for CallChat messaging and calls.

Data and code availability

The paper, claim matrix, structural model, source-hash ledger, and test summary are released in the ResearchForumOnline research repository. Private keys, credentials, customer data, production messages, server topology, entitlement material, and third-party source bundles are excluded.

Conflict of interest

I created and maintain components in the Zmail, ZMath, and CallChat ecosystem. This is not an independent audit.

AI-use disclosure

AI-assisted tools supported corpus search, source comparison, test orchestration, claim-risk review, and editorial refinement. I am responsible for the released evidence, limitations, and conclusions.

References

1. Matrix.org Foundation. *Olm and Megolm*. Matrix Specification 1.18, 2026. <https://spec.matrix.org/v1.18/olm-megolm/>
2. Barnes, R., et al. *The Messaging Layer Security (MLS) Protocol*. RFC 9420, 2023. <https://doi.org/10.17487/RFC9420>
3. Rescorla, E. *WebRTC Security Architecture*. RFC 8827, 2021. <https://doi.org/10.17487/RFC8827>
4. Jones, P., et al. *WebRTC 1.0: Real-Time Communication Between Browsers*. W3C Recommendation. <https://www.w3.org/TR/webrtc/>
5. Dworkin, M. *Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007. <https://doi.org/10.6028/NIST.SP.800-38D>
6. Krawczyk, H., and Eronen, P. *HKDF*. RFC 5869, 2010. <https://doi.org/10.17487/RFC5869>
7. Barker, E., et al. *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. NIST SP 800-56A Rev. 3, 2018. <https://doi.org/10.6028/NIST.SP.800-56Ar3>

Appendix. Snapshot hashes

Artifact	SHA-256
Zmail protected-mail module	01c6442ac4080a64a8641ea657cac872f43514c1a9822b11b5ba1eb7287de7d9
Zmail protected-mail test	f0039be7b67da0a52f3a1e48aa161c57f1fdf2a6dd009e05f286e39da1f9bb19
CallChat media verifier	ed300069eb51fed6c78e7d302f7e94d332e2ea9ad5287e83f73f7f2fcaff2e00
CallChat ZMath Auto module	0f614659ef3e07b90d772057ca4ffadba5808b8c67c5d1c91e9d318bc230802a
CallChat ZMath Auto test	d1938e8895c84fc38351fcdd2bf77f17ccf8c2900b16406d937abd9ccf3245ed